CSCE 411: Design and Analysis of Algorithms

Week 10: Randomized Algorithms

Date: April 7, 2026 *Nate Veldt, updated by Samson Zhou*

Course Logistics

- Reading from Chapters 5 and 11 this week, especially sections on randomized algorithms and hashing
- Homework 6 due Friday

1 The Matrix Verification and Fingerprinting Problem

Consider the problem of checking if two large structures are identical. A deterministic approach might require reading all elements, taking a long time. Instead, we can use *randomization* to map the large structure to a smaller *fingerprint*.

Given two $n \times n$ matrices A, B, and a proposed product C, checking if $A \times B = C$ deterministically takes at least $O(n^\omega)$ time.

Using a random $n \times 1$ vector $r \in \{0,1\}^n$, the randomized fingerprint check is given by:

Let $\mathcal{F}$ be the set of valid runs. The *fingerprint comparison* is:

$$\text{match}(A, B, C, r) = \begin{cases} \\ \\ \text{False} \end{cases}$$

A _____ occurs when $A \times B \neq C$ but the algorithm still outputs True.

Definition A *Monte Carlo* algorithm is a randomized algorithm whose runtime is deterministic, but whose output has a bounded probability of being incorrect.

1.1 Fingerprinting Properties

Probability of Error For any matrices A, B, and C where $A \times B \neq C$, if $r \in \{0,1\}^n$ is chosen uniformly at random,

$$\Pr[A(Br) = Cr] \leq \frac{1}{2}$$

Optimal Substructure for Evaluation

Lemma 1.1. *If r is an $n \times 1$ vector, the matrix-vector multiplications $B \times r$ and then $A \times (Br)$* _____

Las Vegas Algorithms

Lemma 1.2. *: In contrast to Monte Carlo algorithms, a Las Vegas algorithm*

1.2 Error Amplification

Question 1. *Assume we run Freivalds' algorithm k independent times to check if $A \times B = C$. If the algorithm returns True every time, what is the probability that $A \times B \neq C$ (i.e., we are seeing a false positive)?*

A *It will always be exactly* $1/2$

B *It will be at most* $1/k$

C *It will be at most* $(1/2)^k$

D *The probability will drop to exactly 0 after* $k = 2$ *runs*

1.3 Freivalds' Algorithm Basics

Let k denote the number of independent trials. In our algorithm, we maintain the following attributes for each trial i:

Attribute	**Explanation**	**Initialization**	**Invariant**
$trial.r$			
$trial.result$			

Given values for these attributes, we can construct an overall *decision* structure:

- If any trial returns False, the matrices are definitely not equal.

- If all trials return True, we assert they are equal with high confidence.

If $trial.result =$ True for all k trials, then the confidence graph G_c is called a _____

1.4 Generic Fingerprinting Algorithm

The key idea behind randomized fingerprinting is to successively "test" randomly generated fingerprints.

TestFingerprint(A, B, C)

Generate a random vector $r \in \{0,1\}^n$
if $A \times (B \times r) \neq C \times r$ **then**

end if

GenericFreivalds(A, B, C, k)

Initialize-Trials(k)
while We have not exceeded k trials **do**

end while

---

small-graph-fingerprint.png

small-graph-fingerprint.png

small-graph-fingerprint.png

small-graph-fingerprint.png

2 The Universal Hashing Algorithm

Let's do a quick recap of what we have learned about dictionaries and hashing.

Adversarial Inputs Let h be a hash function mapping keys to a table of size m.

- Case 1: h is fixed and deterministic: an adversary can choose n keys that all hash to the same slot, leading to $O(n)$ search time.
- Case 2: h is chosen randomly from a universal family: then the expected number of collisions is bounded, and we have $O(1)$ expected search time.

If Case 1 is true, we would prefer to be aware of this, in which case we state that the worst-case time is $O(n)$ and we're done.
Otherwise, we use randomized hash families when solving the dictionary problem.

```
HashInsert(T, x, h)
  if T[h(x)] is empty then
      T[h(x)] = x
  else
      Append x to the chain at T[h(x)]
  end if
```

```
GenericHashing(S, m)
  Initialize-Hash-Table(m)
  Choose h ∈ H uniformly at random
  while There are elements x ∈ S to insert do
      HashInsert(T, x, h)
  end while
```

Hashing and Generic Algorithm Now we are ready to find a more concrete implementation of this approach, by answering the question:

2.1 Universal Hashing Algorithm Pseudocode

```
UNIVERSAL-HASH-TABLE(S, m, p)
  for i = 0 to m − 1 do
      T[i] = NIL
  end for
  a = RANDOM(1, p − 1)
  b = RANDOM(0, p − 1)

  for x ∈ S do
      h(x) = ((a · x + b) mod p) mod m
      HASHINSERT(T, x, h)
  end for

  for i = 0 to m − 1 do
      if Length(T[i]) > 1 + |S|/m then
          Return "the chain is longer than expected"
      end if
  end for

  V_H = {x ∈ S}
  E_H = {(x, y): h(x) = h(y)}
  Return T and {Length(T[i]): i ∈ {0 ... m − 1}}
```

Question 2. *What is the expected runtime complexity of an unsuccessful search using universal hashing with chaining, where $\alpha = n/m$ is the load factor?*

A $O(1 + \alpha)$

B $O(\log n)$

C $O(n/m + \log m)$

D $O(n)$

```
UNIVERSAL-HASHING(S, m, p)
  INITIALIZEHASHTABLE(m)
  a = RANDOM(1, p − 1), b = RANDOM(0, p − 1)
  for x ∈ S do
      h(x) = ((a · x + b) mod p) mod m
      HASHINSERT(T, x, h)
  end for
  Return T
```

twohash.png

twohash.png

twohash.png

twohash.png

2.2 Correctness

Theorem 2.1. *Assuming that the hash family $\mathcal{H}_{p,m}$ is universal, when* UNIVERSAL-HASHING *is used, the expected length of the chain $E[Length(T[h(x)])]$ will equal $1 + \alpha$ for every $x \in S$.*

2.3 Catching Bad Hash Functions

REHASH-IF-BAD (Collision check)

```
Do everything else...
for i = 0 to m − 1 do
    if Length(T[i]) > 2 · (1 + α) · log n then
        Return an error, the hash function chosen was poor
    end if
end for
```

Theorem 2.2. *If we pick h randomly from a universal family, the probability that the code above produces an error and we need to rehash is very small (by Markov's/Chernoff bounds).*

3 PageRank Algorithm

```
PageRank(G, d, tol)
  Initialize-PageRank(G, d)
  S = ∅
  Q = V
  while diff > tol do
      u = ExtractMaxChange(Q)
      S = S ∪ {u}
      for v ∈ Adj[u] do
          UpdateRank(u, v, d)
      end for
  end while
```

This algorithm maintains:

- $PR[u]$ = current estimate of the stationary probability for node u
- d = damping factor (teleportation probability) to handle spider traps and dead ends

Question 3. *Why do we introduce the damping factor d in the PageRank algorithm?*

A *To speed up the adjacency list traversal*

B *To guarantee convergence by making the transition matrix strongly connected and aperiodic*

C *To eliminate all nodes with in-degree 0*

D *To reduce the memory usage of the algorithm*

3.1 Example

small-graph-pagerank.png

small-graph-pagerank.png

small-graph-pagerank.png

small-graph-pagerank.png

3.2 Correctness

```
PageRank-PowerIteration(G, d, iters)
  Initialize-PageRank(G)
  for i = 1 to iters do
      next_PR = array of zeros
      for u ∈ V do
          next_PR[u] = (1−d)/|V|
          for v ∈ In(u) do
              next_PR[u] += d · PR[v]/Out(v)
          end for
      end for
      PR = next_PR
  end for
```

We will take the following properties as given.

Stochastic property For each column in the modified transition matrix M', the sum of the elements is exactly 1.

Convergence property Let π be the stationary distribution. By the Perron-Frobenius Theorem, iterative multiplication by a positive, column-stochastic matrix M' will converge to the dominant eigenvector.

Teleportation property If there is a spider trap in the graph, the $1 - d$ probability of teleporting ensures the random surfer does not get permanently stuck.

Theorem 3.1. *PageRank algorithm, run on a directed web graph $G = (V, E)$ with damping factor $d < 1$, terminates with $PR[u] = \pi(u)$ for all $u \in V$, where $\sum \pi(u) = 1$.*

Proof. We maintain the following invariant throughout.

Invariant: At the start of the while loop, the vector PR satisfies $\sum_{u \in V} PR[u] = 1$.

If this invariant is true, the theorem is shown. Why?

Assume this invariant is *not* true and we will show a contradiction.

Define t to be the first iteration where the sum of probabilities PR_t is not 1.

1. We know $t > 0$, since PR_0 is initialized to $1/|V|$ for all u.

2. We know $\sum_{u \in V} PR_{t-1}[u] = 1$.

3. Let M' be the modified transition matrix combining the web graph and teleportation.

4. We know that $PR_t = M' \times PR_{t-1}$

5. We then know $\sum_u PR_t[u] = \sum_u \sum_v M'_{uv} PR_{t-1}[v]$

6. We know that this can be rewritten as $\sum_v (PR_{t-1}[v] \sum_u M'_{uv})$

7. And we know $\sum_u M'_{uv} = 1$ because M' is column-stochastic

8. We know also that $\sum_v PR_{t-1}[v] = 1$ from our assumption

9. The last steps show $\sum_u PR_t[u] = \sum_v PR_{t-1}[v] \cdot 1 = 1$. Why is this a contradiction?

3.3 Runtime Analysis

```
INITIALIZE-PAGERANK(G)
for i = 1 to iters do
    next_PR = array of zeros
    for u ∈ V do
        next_PR[u] = (1-d)/|V|
        for v ∈ In(u) do
            next_PR[u] += d · PR[v]/Out(v)
        end for
    end for
    PR = next_PR
end for
```

Question 4. *Assuming the graph is stored using an adjacency list containing incoming edges for each node, what is the runtime of a single iteration of PageRank? (Warning: there may be more answer that you can argue is valid! Don't just select an answer, think about a specific reasoning for the runtime analysis you select).*

A $O(V + E)$

B $O(V^2)$

C $O(VE)$

D $O(E \log V)$

E $O(E + V \log V)$

3.4 Class Activity: another example

pagerankalg.png